

— Web apps

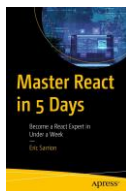
Chapter 1: React components

L. Espeel



— Leermateriaal

- Handboek: **Master React in 5 Days: Become a React Expert in Under a Week**



Sarrion, E. (2023). Master React in 5 Days: Become a React Expert in Under a Week (1st ed. 2023.). Apress.

E-book beschikbaar op <https://limo.libis.be/VIVES>

<https://doi-org.vives.e-bronnen.be/10.1007/978-981-19-1800-1>

- Slides: extra notities en aanvullingen

— Overzicht hoofdstukken

Boek:

1. React components
2. JSX code
3. Event handling
4. React Hooks
5. Application: a task list with React
6. JavaScript reminders

Extra:

7. React routes
8. PWA – introduction
9. React Native – introduction

3

— Inhoud

- In dit hoofdstuk:
 - Why use React
 - React Virtual DOM
 - Creating the Virtual DOM, Reconciliation Process
 - Decomposing an Application into Components
 - Analyzing the Main Files of the React Application
 - *index.js*, *index.html*, *App.js*
 - JSX Syntax in React
 - Creating a First React Component
 - Installation, code example counter, *useState()*, *useEffect()*, using attributes

4

— React

- Wat?
 - Open source JavaScript library ontwikkeld door Meta (Facebook) om interactieve user interfaces te maken voor Web Apps, vooral voor single-page applicaties (SPA's).
- Belangrijkste kenmerken:
 - Herbruikbare componenten
 - Virtual DOM: representatie van de UI (HTML-elementen) in het geheugen
 - Documentatie en grote community die meebouwt aan React
 - Multi-platform: Web, mobile, desktop
 - React en React Native (voor native iOS en Android apps)
 - Uitgebreid ecosysteem: Redux, GraphQL, Next.js, Gatsby, Material UI

5

— React Virtual DOM

- Representatie van de UI in het geheugen
 - Waarom? Rechtstreekse DOM-aanpassingen zijn te traag als React-component wordt aangepast.
- Bij aanpassing aan een React-component:
 - React update Virtual DOM, dit gaat sneller.
 - React voert reconciliatie (render) uit.
 - Vergelijken van oude Virtual DOM ↔ nieuwe Virtual DOM:
→ Alle wijzigingen in één keer aan DOM aanbrengen.

```
<div>
  <h2> Counter: 0</h2>
  <button>Increment</button>
</div>
```

```
<div>
  <h2> Counter: 1</h2>
  <button>Increment</button>
</div>
```

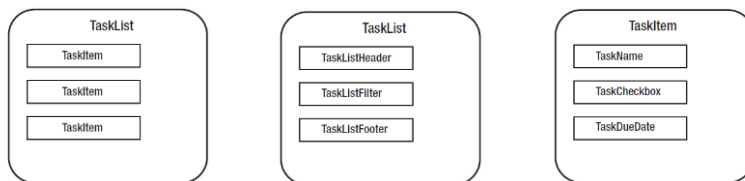


Counter: **1**

6

— Applicatie verdelen in componenten

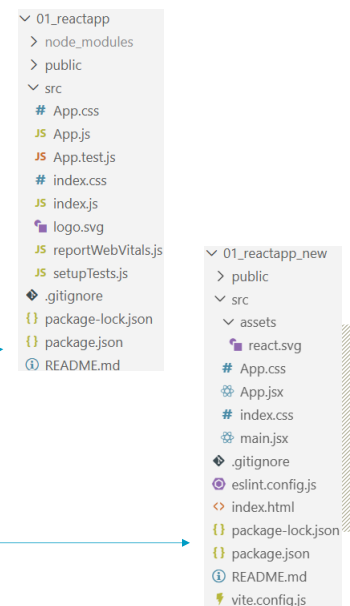
- Het opbouwen van de code met componenten zorgt voor:
 - Begrijpbare code
 - Onderhoudbaarheid (mogelijk om aanpassingen te doen)
 - Herbruikbare componenten
 - Debugbaarheid: elke component is apart testbaar



Bron: Sarrion, E. (2023). *Master React in 5 Days* (1st ed. 2023.). Apress.

— Een eerste React-applicatie (1)

- Nodig:
 - Editor zoals Visual Studio Code
 - Node.js
 - <https://nodejs.org/dist/v22.19.0/node-v22.19.0-x64.msi>
 - Gebruik commandoventer om een React-applicatie aan te maken:
 - Legacy manier (volgens boek, zullen wij ook gebruiken):
 - `npm install -g create-react-app`
 - `create-react-app <name>`
 - `cd <name>`
 - `npm start`
 - Browser open en surf standaard naar <http://localhost:3000>
 - Nieuwere manier met Vite build tool:
 - `npm create vite@latest <name> -- --template react`
 - `cd <name>`
 - `npm run dev`
- Open een browser en surf standaard naar <http://localhost:5173>



— Een eerste React-applicatie (2)

- App componenten → render de App-component in het root-element:

- index.html*

```
31 | | <div id="root"></div>
```

- index.js* (of *main.jsx*):

```
1 import React from 'react';
2 import ReactDOM from 'react-dom/client';
3 import './index.css';
4 import App from './App';
5 import reportWebVitals from './reportWebVitals';
6
7 const root = ReactDOM.createRoot(document.getElementById('root'));
8 root.render(
9   <React.StrictMode>
10   | <App />
11   </React.StrictMode>
12 );
```

React.StrictMode: laat toe om info naar console weg te schrijven bij fouten.
Meer info: <https://react.dev/reference/react/StrictMode>.

— Een eerste React-applicatie (3)

- App.js* (*):

(*) indien gebruik gemaakt werd van de Vite build tool, is de extensie van *.js* standaard *.jsx*.

```
1 import logo from './logo.svg';
2 import './App.css';
3
4 function App() {
5   return (
6     <div className="App">
7       <header className="App-header">
8         <img src={logo} className="App-logo" alt="logo" />
9         <p>
10           Edit <code>src/App.js</code> and save to reload.
11         </p>
12         <a
13           className="App-link"
14           href="https://reactjs.org"
15           target="_blank"
16           rel="noopener noreferrer"
17         >
18           Learn React
19         </a>
20       </header>
21     </div>
22   );
23 }
24
25 export default App;
```

— JSX-syntax

- JSX = JavaScript and XML
- Componenten aanmaken zoals in HTML: JSX-code wordt omgezet naar JavaScript object.
- Variabelen staan tussen accolades:

```
import React from "react";
function App() {
  const name = "John Doe";
  const greeting = <h1>Hi, {name} !</h1>;
  return (
    <div>
      {greeting}
      <p>This is an example of JSX code in React.</p>
      <button>Click</button>
    </div>
  );
};
export default App;
```

11

— Een eerste React-component (1)

- Voorbeeld: **teller** maken dat elke seconde incrementeert.

- *App.js* aanpassen naar:

```
3 import Counter from "./Counter.js"

5 function App() {
6   return (
7     <Counter />
8   );
9 }
```

- Extra bestand *Counter.js* aanmaken:
- In *App.css* kan je eventuele extra styling toevoegen.

```
1 function Counter() {
2   let count = 0;
3   return (
4     <>
5       The counter is set to: {count}
6     </>
7   )
8 }
9
10 export default Counter;
```

12

— Een eerste React-component (2)

- Console-venster bekijken kan in de meeste browsers via de F12-toets.
 - Uitvoer van DOM (onder tabblad *Elements* in Chrome).
- Er wordt gesuggereerd om React DevTools extensie te installeren, met dit zie je de hiërarchie van de React-componenten en hoelang ze nodig hebben om te laden. Interessant om te debuggen.
 - Eventueel moet je browser opnieuw opstarten na installatie extensie.
 - Extra tabbladen zijn op het einde in het console-venster terug te vinden: *Components*, *Profiler*

- Counter-functie verder uitbreiden met *setInterval*-functie:

```
1 function Counter() {  
2   let count = 0;  
3  
4   setInterval(function () {  
5     count++  
6     console.log("count =", count);  
7   }, 1000);  
}
```

13

— Een eerste React-component (3)

- Uitvoer: waarde in console telt op, maar op pagina blijft het 0 ...
 - Reden: React weet niet dat de toestand (state) veranderd is en een reconciliatie (render) nodig is.
- En ... een render-functie moet "pure" zijn:
 - Het mag geen "side effects" bevatten → geen aanmaak van timer toegelaten.
 - Het moet deterministisch zijn: steeds hetzelfde resultaat geven met zelfde parameters.

14

— Een eerste React-component (4)

- Oplossing: gebruik maken van **reactive** variabelen.
 - Bij wijziging van zo'n variabele → React rendert component zo snel mogelijk.
- React heeft concept van **state**:
 - Is een set van *reactive* variabelen.
 - Gebruik de *useState()*-functie (wordt ook **hook** genoemd) om een *reactive* variabele te maken.
- Aanpassing *Counter.js*:

```
1 import { useState } from "react";
2
3 function Counter() {
4   const [count, setCount] = useState(0);
5
6   setInterval(function () {
7     setCount(count + 1);
8     console.log("count =", count);
9   }, 1000);
```

Equivalent aan:
const stateCount = useState(0);
const count = stateCount[0];
const setCount = stateCount[1];

— Een eerste React-component (5)

- Bekijk de uitvoer, zowel in console als webpagina. Wat merk je op?
- Oorzaak: *setInterval* wordt bij elke reconciliatie (render) opgeroepen. Dit zorgt voor meerdere actieve timers.
- Oplossing: gebruik de *useEffect()*-functie.
 - **Setup**: deze functie wordt uitgevoerd bij elke reconciliatie (render).
 - **Cleanup**: deze functie wordt gerouterneerd en wordt vóór een nieuwe reconciliatie (render) uitgevoerd.

Import aanpassen naar:

```
1 import { useState, useEffect } from "react";
```

— Een eerste React-component (6)

```
14     useEffect(function () {
15         const timer = setInterval(function () {
16             setCount((count) => count + 1);
17             console.log("count =", count);
18         }, 1000);
19         return function () {
20             clearInterval(timer);
21         }
22     });
```

Setup

Cleanup

- Uitvoer? Vaststelling tussen console en webpagina?
- Kopieer nu eens regel 16 en plaats deze op regel 17 (m.a.w. 2x `setCount(count + 1)` aanroepen). Vaststelling: teller blijft per 1 tellen ... Waarom?
 - Oplossing: gebruik `setCount((count) => count + 1)`;

17

— Een eerste React-component (7)

- Er kunnen ook **attributen** gebruikt worden in een component.
 - Bv. `init`-attribuut meegeven in `App.js`:
`<Counter />`
`<Counter init="5" />` // 1) als stringwaarde
`<Counter init={5} />` // 2) als JavaScript-waarde
 - Het attribuut wordt via `props` opgevraagd in `Counter.js`:

```
3     function Counter(props) {
4         const init = parseInt(props.init || 0); // parseInt is optioneel indien 2) gebruikt wordt
5         const [count, setCount] = useState(init);
6
7         useEffect(function () {
8             // ...
9
10            // ...
11
12            // ...
13
14            // ...
15
16            // ...
17            return (
18                <>
19                    Initial value of the counter is: {init}
20                    <br />
21                    The counter is: {count}
22                </>
23            );
24        });
25    }
```

18

— Een eerste React-component ⁽⁸⁾

- Opmerking: als je letterlijk `<Counter init="5" />` wilt afdrukken naar de browser, dien je dit toe te passen in de return-blok (in *App.js*):

```
return (  
  <>  
    Counter defined by: {<Counter init='5' />}  
    <br />  
  </>  
)
```